

Slurm AI Agent

面向 USTC 107 算力平台的智能作业助手

设计文档

从自然语言意图到可验证 Slurm 作业的闭环设计

文档版本： V1.0

项目版本： 基于 2026-09-03 源码快照

技术栈： FastAPI / OpenAI SDK / Slurm REST API / WebSocket

部署目标： USTC 107 算力平台登录节点

摘要

Slurm AI Agent 是一套部署在共享 HPC 登录节点上的智能作业工作台。系统将项目文件、隔离环境、资源选择、作业提交、运行监控、日志分析和平台知识问答整合到一个 Web 界面，并通过大模型 Function Calling 将自然语言请求转换为可审计、可校验的工具调用。其设计重点不在于让模型直接执行任意命令，而在于把大模型限定在“理解与编排”层，把权限、资源、路径和提交过程固定在确定性后端中，形成从项目准备到结果复盘的安全闭环。

目录

1	项目概述	3
1.1	背景与痛点	3
1.2	设计目标：从痛点到对策	3
1.3	设计范围与非目标	4
2	总体设计思路	4
2.1	从“聊天机器人”到“受控执行代理”	4
2.2	项目化状态模型	4
3	技术架构	5
3.1	分层架构	5
3.2	核心组件职责	5
3.3	外部接口与通信协议	6
4	关键业务流程	6
4.1	智能对话与工具调用	7
4.2	依赖识别、确认与安装	7
4.2.1	多源确定性扫描	7
4.2.2	版本感知预检	8
4.2.3	AI 复核的权限边界	8
4.2.4	状态版本化与并发互斥	8
4.3	受控作业提交	8
4.4	监控与结果分析闭环	9
5	功能模块设计	9
6	模型与 API 调用设计	10
6.1	使用的模型	10
6.2	调用方式	10

6.3 工具能力矩阵 11

7 数据与状态设计 11

8 安全性与可靠性设计 11

8.1 安全边界 11

8.2 可靠性机制 12

9 技术难点与解决方案 12

9.1 异构 Slurm 接口语义对齐 12

9.2 模型生成与真实执行之间的可信鸿沟 12

9.3 依赖安装引擎的失败分类与分层自愈 13

9.4 共享登录节点上的环境与并发 14

9.5 跨生命周期上下文闭环 14

9.6 浏览器终端的系统编程细节 14

10 创新点 14

11 部署与运行设计 15

11.1 运行环境 15

11.2 启动与访问链路 15

12 测试与验收建议 15

13 可演进方向 16

14 结论 17

1 项目概述

1.1 背景与痛点

Slurm 是科研计算和深度学习训练中常用的集群调度系统，但对初学者并不友好：分区、账户、QoS、TRES、作业脚本、环境管理和日志路径，任何一处都可能成为第一次提交的障碍。传统命令行流程还把任务上下文分散在文件系统、终端、调度队列和文档之间。归纳起来，痛点集中在五个方面：

- 1. **资源配置易错**：参数组合繁多，写错即提交失败，失败原因还常常不直观。
- 2. **依赖环境易坏**：版本冲突、来源混装、误装进 base 环境，排查成本高。
- 3. **上下文分散**：代码、数据、环境、命令、日志和文档各在一处，任务被打断后难以接续。
- 4. **失败难以复盘**：排队原因无解释，报错堆栈不友好，日志位置靠记忆。
- 5. **共享节点安全顾虑**：多用户环境下，开放端口、明文密钥和不受限的命令执行都是风险。

本项目面向 USTC 107 算力平台，把这五个痛点转化为一条项目级 workflow 逐项应对。用户仍然决定安装哪些依赖、申请多少资源以及何时提交；系统负责采集真实上下文、提供建议、执行校验并持续跟踪结果。这一“人做决策、模型做理解、程序做约束”的职责划分，是整体设计的核心。

1.2 设计目标：从痛点到对策

痛点	设计目标	设计对策
资源配置易错	真实可验证	资源与作业状态一律来自实时接口；提交前做授权、配额与路径校验，提交后回读 TRES/GRES 核验 GPU 请求是否生效。
依赖环境易坏	可控安装、可自愈	依赖清单由确定性扫描与真实软件源预检生成，用户确认后才执行；安装命令规范化并互斥运行，中断留下的残缺环境可自愈重建。
上下文分散	上下文一致	以项目为根组织代码、数据、环境、会话与作业记录；文件一旦变更即标记依赖分析失效，避免复用过期结论。
失败难以复盘	结果可追溯	提交后的作业注册为观察记录，终态自动聚合 Slurm 状态与日志生成报告；失败中的缺失依赖写回需求，反哺下一轮检查。
共享节点安全顾虑	安全可控	密钥不写入代码、网络默认隧道访问、CLI 白名单；安装、提交等有副作用的操作均由用户明确触发。

贯穿上述对策的手段是自然语言交互：模型负责理解与解释，降低 Slurm 概念的学习成本，但不改变表中任何一条约束。另有一项横切目标是**可降级**：Embedding 或模型分析失败时，分别退回关键词检索或原始日志报告，保证核心功能在外部服务异常时不中断。

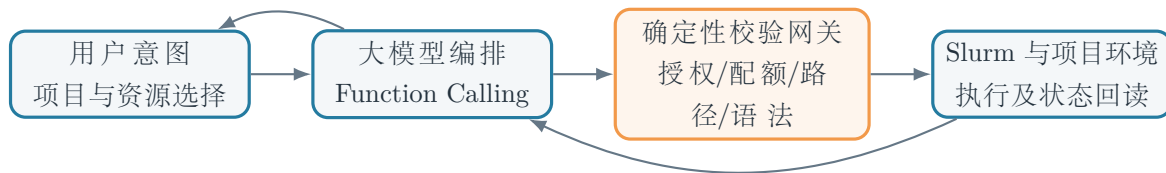
1.3 设计范围与非目标

系统覆盖 Web 工作区、智能体、多源 Slurm 能力、依赖规划、作业模板、受控提交、监控分析、文件管理和登录节点终端。不将大模型作为授权主体，不允许其自由拼装写操作 CLI；不替代 Slurm 调度器，也不承诺改变排队优先级；交互式 `srun --pty` 等需真实 TTY 的工作流仍由用户在终端完成。

2 总体设计思路

2.1 从“聊天机器人”到“受控执行代理”

系统把任务拆为三类能力：大模型负责语义理解、步骤规划和结果解释；工具层负责将结构化意图映射到有限能力；确定性后端负责权限、参数、路径、环境和副作用控制。模型的输出不是最终提交脚本，而是结构化资源字段和命令正文。服务端随后生成锁定的脚本前导、工作目录、Conda 激活和日志配置，从而避免提示注入或模型幻觉直接改变调度边界。

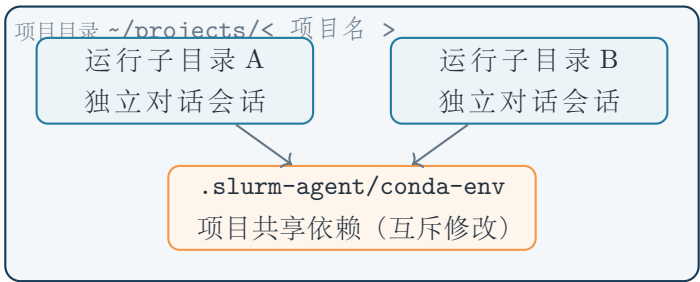


2.2 项目化状态模型

默认工作空间为 `~/projects/<项目名>`。每个项目共享一个 `.slurm-agent/conda-env`，可包含多个运行子目录；每个运行目录维护独立对话上下文，同时共享项目依赖。这种模型兼顾实验隔离和依赖复用，也便于把提交脚本、标准输出、错误日志和报告统一放入 `logs/`。

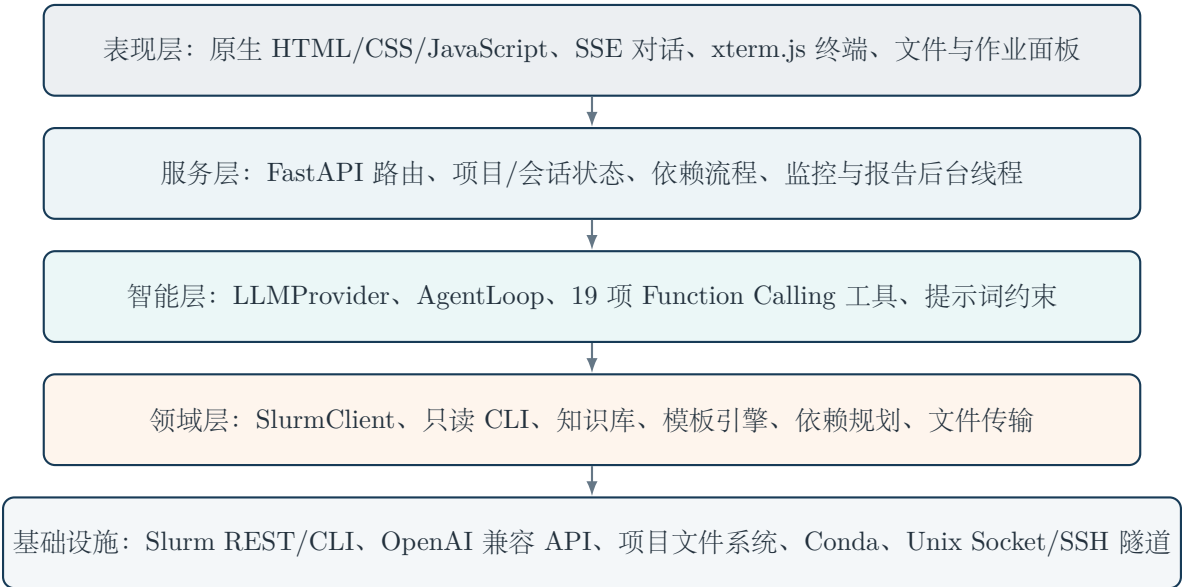
```
~/projects/<project>/
|-- activate.sh
|-- <source, data, config>
|-- <run-subdir>/
|-- logs/
|-- runs/
`-- .slurm-agent/
    |-- conda-env/
    |-- dependency-analysis.json
    `-- install.lock
```

目录结构背后的共享关系如下图：多个运行子目录各自维护独立对话上下文，但激活同一个项目级 Conda 环境，安装与修改受互斥锁保护。



3 技术架构

3.1 分层架构



3.2 核心组件职责

组件	实现位置	主要职责
Web/API 服务	server/app.py	49 个 HTTP/WebSocket 端点，组织项目、模型、仪表盘、依赖、作业、报告、文件等业务流程。
智能体循环	agent/agent_loop.py	维护消息历史，接收模型工具调用，逐项执行并回填 tool 消息，直到模型产生最终文本。
模型适配	agent/llm_provider.py	使用 OpenAI Python SDK 调用兼容接口，动态读取当前模型，支持工具定义、温度和输出长度控制。
工具注册表	agent/tools_registry.py	声明 19 项工具的 JSON Schema，并把工具名路由到 REST、CLI、知识库、模板或项目文件能力。

组件	实现位置	主要职责
Slurm 客户端	core/slurm_client.py	封装 slurmrestd v0.0.41，请求头认证、401 自动刷新重试、错误规范化、提交与取消。
只读 CLI 层	core/cli_executor.py	补足 REST 未覆盖的优先级、fairshare、实时占用、用量报表和账户授权查询。
知识库	core/knowledge_base.py	Markdown 按标题分块，Embedding 批量向量化、磁盘缓存、余弦检索和关键词降级。
依赖规划	core/dependency_planner.py	扫描多种依赖文件与源码 import，版本预检、合并去重、安装计划生成和安装后验证。
文件与环境	core/file_transfer.py	工作区创建、安全相对路径、上传限制、项目级 Conda 和跨线程/进程安装锁。
终端桥接	server/terminal.py	WebSocket 双向字节流连接 PTY/bash，处理尺寸变化、并发限制、Origin 校验与断连清理。

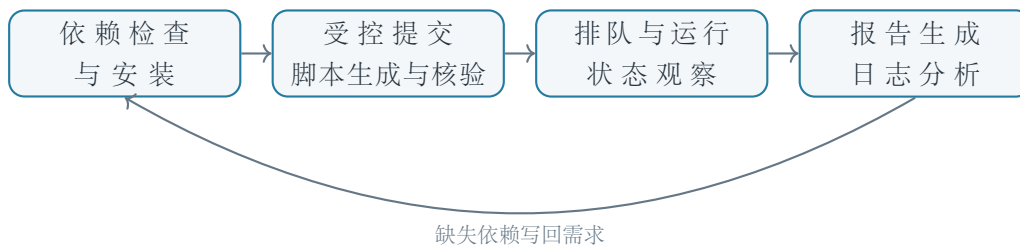
3.3 外部接口与通信协议

接口	协议与地址	说明
模型对话	OpenAI 兼容 POST /chat/completions	基址 https://api.llm.ustc.edu.cn/v1 ； 携带 tools 时启用 Function Calling； Key 仅取自 LLM_API_KEY。
文本向量	OpenAI 兼容 POST /embeddings	同一基址；用于知识库分块向量化与 查询向量生成。
Slurm REST	http://107.ustc.edu.cn:6820 ，前缀 /slurm/v0.0.41	请求头 X-SLURM-USER-TOKEN 携带 JWT；历史能力使用 slurmdb 前缀； 401 自动刷新后重试一次。
浏览器—服务	JSON HTTP / text/event-stream / WebSocket	普通业务用 HTTP；聊天与安装进度 用 SSE；交互终端用 WebSocket 二 进制帧传 PTY 字节。
服务入口	Unix Domain Socket + SSH LocalForward	Uvicorn 绑定权限受限的 Socket，浏 览器经隧道访问，不在共享登录节点 公开监听 TCP 端口。

4 关键业务流程

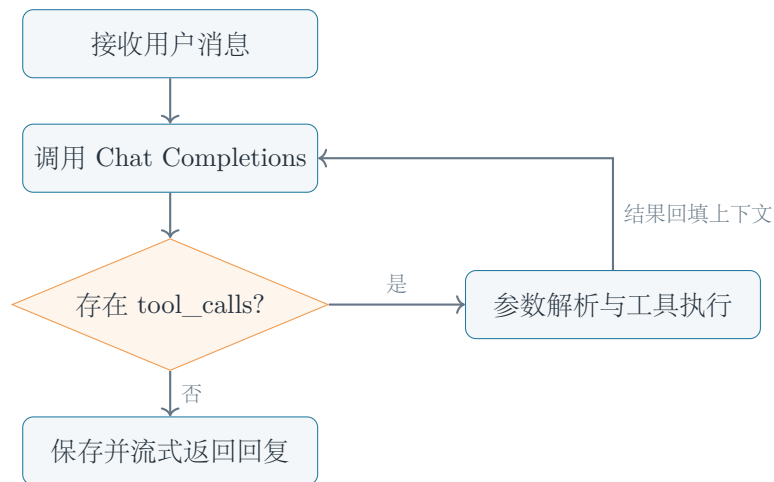
四个核心流程按作业生命周期依次展开：智能对话贯穿全程，依赖流程发生在准备阶段，受控提交把准备成果落为调度实体，监控分析再把运行结果反馈回项目——最后一环回到依赖，形

成闭环。



4.1 智能对话与工具调用

用户消息进入项目对应的 Agent 实例。模型收到 system prompt、历史消息和工具 Schema 后，可返回零个或多个工具调用；执行器将工具结果作为 `role=tool` 消息加入上下文，再次请求模型，直到得到自然语言回复。项目会话、文档问答和文件管理问答相互隔离，降低跨场景上下文污染。



4.2 依赖识别、确认与安装

依赖流程不直接让模型生成并执行命令，而是遵循“确定性扫描打底、真实软件源预检、模型仅复核补充、用户最终确认”的分层设计。强调确定性优先，原因很直接：清单上的每一项最终都会变成一次真实的软件安装，一个被模型幻觉出来的包名或版本号，代价往往是几十分钟的失败排障。

4.2.1 多源确定性扫描

扫描器遍历项目目录，识别六类可信来源：`requirements.txt`、`environment.yml`、`pyproject.toml`、`Pipfile`、`setup.py` 的显式声明，以及 Shell 脚本（`.sh/.bash/.sbatch`）和 Markdown 文档中的安装命令与依赖清单。对 README 这类自由文本，扫描器同时识别三类内容：行内或代码块中的 `pip install / conda install` 命令、“Requirements/依赖”标题下的列表项，以及代码块中连续出现的“单包名 + 版本约束”清单（连续三行以上才采纳，避免把 Shell 片段误判为清单）；并用包名黑名单过滤“Install xxx with conda”这类说明性句子，防止把动词当成包名。

显式声明之外，扫描器用 AST 解析全部 Python 源码的 `import` 语句：先剔除标准库与项目本地模块（含 `src-layout` 与命名空间包），再经一张刻意保持精简的稳定别名表（如 `cv2` → `opencv-python`、`yaml` → `PyYAML`）映射为发行包名。无法归入显式依赖的 `import` 被标记为“待模型复核”的候选，并保留文件、行号和语句原文作为证据。用户补充的文字需求（如“需要 `gromacs` 2026.3”）也被确定性解析为依赖项——含用户输入的安装命令、常见软件别名表和邻近版本号提取——不依赖模型推断。

4.2.2 版本感知预检

预检依次回答三个问题。其一，环境里是否已装过：一次 `conda list` 快照，已装项默认不勾选，避免重复安装。其二，软件源里是否真的有这个包：`pip` 走 `pip index versions`，`conda` 走 `conda search --json` 并锁定渠道，结果带 10 分钟 TTL 缓存，`report` 与 `install` 两个阶段复用，避免重复下载 `reodata`。其三，请求的版本是否存在：按 PEP 440 规则解析版本约束（含 `conda version=build` 三段式的转换），与真实可用版本逐一比对。

版本不存在时，系统标记 `version_mismatch` 并给出建议版本：优先取同 `major.minor` 的最新版，其次同 `major`，最后才是软件源最新版；`conda` 包还会展示匹配范围内最新的构建变体（如 `nompi/cuda`）。这一步专门防范一类高频错误：把其它集群 `module` 系统里的版本号（如 `gromacs/2019.4-gcc-9.2.0` 中的 2019.4）直接当成 `conda` 可安装版本。此外，`conda 25.x` 首次访问软件源需接受 ToS，系统在分析与安装前自动代为接受（幂等操作），用户无需感知。

4.2.3 AI 复核的权限边界

只有预检后仍未解决的 `import` 候选会交给模型分类，动作限定为外部依赖、已被显式依赖覆盖、可选、忽略、不确定五种。模型无权修改显式声明：即使模型声称某 `import` “已被覆盖”，系统也要求其指出的提供包确实存在于显式依赖列表中，否则保留候选——模型不能凭空删除依赖。所有 AI 建议默认不勾选，装什么、不装什么由用户在确认弹窗中最终决定。

4.2.4 状态版本化与并发互斥

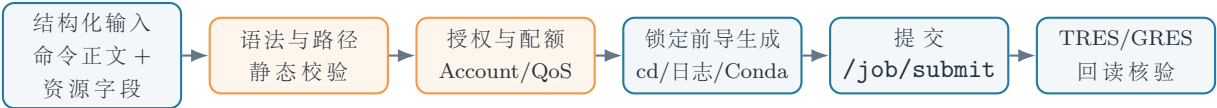
依赖状态文件含 `revision` 与 `analyzed_revision`。文件上传或编辑会把项目标记为 `dirty`，避免复用过期分析结果；缓存命中时仅做一次轻量 `conda list` 刷新已装状态，不重新扫描。同一项目使用线程锁与 `flock` 双层互斥，防止多个请求同时修改 Conda 前缀；中断产生的半成品环境会在重建前清理。

4.3 受控作业提交

1. 用户或模型提供项目、运行子目录、命令正文和结构化资源字段。
2. 服务校验命令不得含 `shebang` 或 `#SBATCH`，并限制长度；校验作业名与资源参数的整数范围，并检查 CPU 分区上是否误申请了 GPU。
3. 实时读取用户 Account/QoS 授权与分区允许范围，结合 REST 返回或静态兜底的 QoS 上限校验 CPU、GPU、内存和时限。

4. 使用 `shlex` 解析逻辑命令，验证 Python/Shell 入口、配置、权重和断点路径；识别重复运行目录前缀。
5. 后端锁定生成 `set -euo pipefail`、`cd`、日志目录和项目 Conda 激活，再调用 `/job/submit`。
6. 取得 `job_id` 后立即查询作业详情，从 TRES/GRES 证据核验 GPU 数；证据不足时明确标记 `pending`，绝不把“已提交”误报为“GPU 已确认”。

整条提交链路如下图，橙色环节为校验网关，任一不通过即拒绝并指出具体字段：



4.4 监控与结果分析闭环

提交成功的作业写入 `watch` 记录。状态变为终态后，后台线程读取 Slurm 信息和 `stdout/stderr` 尾部。成功作业生成结果概况、关键指标与警告；失败作业生成关键错误、原因和修复步骤。若 LLM 不可用，仍生成包含原始上下文的兜底报告。系统还通过确定性正则提取 `ModuleNotFoundError`、`ImportError` 和 `command not found` 中的缺失依赖，写回项目需求记录，使后续依赖检查能够吸收运行反馈。

5 功能模块设计

下表按用户可见的模块分组，并标注各自的关键设计；其中环境与依赖、作业提交、监控与报告三个模块的实现细节，分别对应第 4 节的关键业务流程与第 9 节的技术难点。

模块	核心功能	关键设计
资源与作业看板	节点、CPU/GPU 状态、实时作业、个人历史作业、提交选项	实时数据与累计统计分口径；后端统一摘要多版本 API 字段。
项目工作区	创建项目、运行子目录、对话历史、上传、文件树	根路径约束、安全相对路径、内部目录保护、2 GB/1000 文件默认上传上限。
智能助手	集群查询、知识问答、文件阅读、脚本生成、日志诊断	19 个受限工具；实际运行数据必须工具获取；项目上下文注入。
模型管理	获取可用模型、刷新、切换	仅保留 <code>deepseek/glm</code> 家族；本地只存模型元数据和 Key 指纹，不存明文 Key。
环境与依赖	项目级 Conda、依赖扫描、选择安装、进度展示、结果验证	Conda/Pip 命令规范化；安装互斥；SSE 分阶段反馈；分析缓存版本化。
模板与作业规划	单卡 PyTorch、多卡 DDP、CPU 批处理、Job Array、Jupyter、通用脚本	JSON 模板参数默认值、范围校验和变量替换；支持个人模板。

模块	核心功能	关键设计
作业提交	Web/Agent 共用提交入口，生成脚本并调用 Slurm	结构化资源、授权校验、路径静态检查、锁定前导、提交后资源核验。
监控与报告	终态检测、日志聚合、成功/失败报告、输出浏览和下载	后台分析不阻塞请求；大文件尾部预览；外部结果链接保留。
文件管理器	树状浏览、编辑、预览、下载、上传、新建、移动、复制、删除	路径归一化、可编辑类型与大小限制、关键内部路径保护。
Web 终端	浏览器内登录 shell、ANSI 显示、窗口缩放	PTY 字节桥接、同源校验、最多 4 会话、SIGHUP 到 SIGKILL 的清理策略。
平台文档	文档树、正文阅读、独立问答	本地 Markdown 为事实来源，Embedding 检索失败时自动转关键词检索。

6 模型与 API 调用设计

6.1 使用的模型

- **默认对话模型**：deepseek-v4-flash，运行时可从当前 API Key 返回的 deepseek/glm 模型中切换。
- **向量模型**：qwen3-embedding，用于平台 Markdown 文档的向量化与查询向量生成。
- **预留重排序模型**：qwen3-reranker，配置已保留但当前检索流程未启用，文档量扩大后可接入。

6.2 调用方式

对话模型使用 OpenAI 兼容 Chat Completions API。请求包含 `model`、`messages`、`temperature`、`max_tokens`，需要工具时附加 OpenAI function tool Schema。模型返回 `tool_calls` 后，由 Agent-Loop 执行并以对应 `tool_call_id` 回填结果。Embedding API 按 16 个片段一批请求，将向量缓存为 NumPy 文件，并以余弦相似度取最多 3 条结果，阈值为 0.3。

```
client = OpenAI(api_key=os.environ["LLM_API_KEY"],
                base_url="https://api.llm.ustc.edu.cn/v1")
response = client.chat.completions.create(
    model=selected_model,
    messages=messages,
    tools=tool_definitions,
    temperature=0.3,
    max_tokens=2048,
)

vectors = client.embeddings.create(
```

```
model="qwen3-embedding", input=batch
)
```

6.3 工具能力矩阵

类别	工具能力	数据/执行通道
作业与集群	实时/单作业查询、提交、取消、诊断、节点、QoS、历史	Slurm REST
调度诊断	优先级、fairshare、实时占用、用量报表、本人授权	只读 Slurm CLI
知识与脚本	文档检索、模板列表、脚本生成	本地知识库/模板
项目上下文	文件树、按行读取文件	受限项目文件系统

7 数据与状态设计

系统没有引入中心数据库，而是利用用户项目目录保存与计算任务天然共址的轻量状态。项目对话、依赖分析、作业观察记录、生成脚本和报告均以 JSON、Markdown 或文本文件持久化，与代码和数据放在同一目录树下。这降低了登录节点部署复杂度，也使用户可直接检查和迁移结果。主要状态如下表：

状态类别	存放内容	载体
模型配置	base URL、Key 指纹、可用模型家族、选中模型、更新时间；不保存密钥	JSON
依赖状态	版本号、项目 revision、最近分析 revision、结构化依赖项和报告	项目目录内 JSON
会话状态	按项目与运行子目录保存角色和文本消息，服务重启后可恢复	JSON
作业观察	job_id、项目、运行目录、日志目录、最终状态、分析状态和报告位置	JSON
知识索引	文档分块元数据与向量缓存	JSON 与 .npy

8 安全性与可靠性设计

8.1 安全边界

系统把安全约束拆成六层边界，模型与前端均无权跨越：

约束与手段	
边界	
密钥	LLM Key 和 Slurm JWT 只从环境变量或本地 .env 读取，不写入代码；界面仅显示脱敏 Token 预览。
网络	Unix Socket 权限配合 SSH 隧道，将 Web 访问限定在当前登录用户，不直接开放共享节点端口。
命令	CLI 仅允许 sprio、sshare、sstat、sreport 和 sacctmgr show/list；argv + shell=False，参数做字符白名单与伪选项拦截。
文件	拒绝绝对路径、...、内部状态目录和越界目标；项目名清洗后再 resolve 并验证根路径前缀。
副作用	安装、提交、取消由用户明确触发；Agent 提交必须绑定项目级 handler，不能从命令行默认会话绕过。
资源	结构化整数范围、账户/分区/QoS 交叉授权和限额校验；GPU 进入 REST job description 而非只写脚本注释。

8.2 可靠性机制

Slurm 请求设置统一超时并规范化 401/404/网络异常；401 最多刷新 JWT 后重试一次。知识检索、结果分析和 QoS 限额读取均设计了降级路径。长耗时的 Conda 初始化、依赖安装和作业报告采用后台执行或 SSE，不阻塞主请求。终端断连时先发送 SIGHUP，宽限后强制终止，避免僵尸 shell。多处输出设置大小或条数上限，控制浏览器、服务和模型上下文的资源消耗。

9 技术难点与解决方案

9.1 异构 Slurm 接口语义对齐

Slurm REST、slurmdbd 和 CLI 的字段、可用性与统计口径并不完全一致。例如 v0.0.41 的 partition 查询参数可能被忽略，系统在客户端二次过滤；提交端点使用单数 /job/submit，nodes 需字符串类型，time limit 字段和 environment 也有平台特定要求。系统把这些差异封装在 Slurm-Client 中，并以 CLI 只读能力补足 REST 空白，避免上层业务同时理解多套语义。

9.2 模型生成与真实执行之间的可信鸿沟

模型能够生成看似正确但路径不存在、资源字段缺失或夹带 #SBATCH 的命令。项目采用“双阶段生成”：模型只给正文，后端锁定关键部分；再进行命令语法和文件存在性检查。提交成功也不等于资源申请生效，因此系统查询作业详情并解析 TRES/GRES 作为证据。这使“自然语言生成”具备可验证的落点。

9.3 依赖安装引擎的失败分类与分层自愈

依赖检查解决了“装什么”，安装引擎要解决“装失败之后怎么办”。科学计算依赖的安装失败是常态而非例外：版本号在软件源中不存在、求解器冲突、源码包缺编译器、大体积下载中断，每一类失败的正确处置方式都不同。系统没有把这些判断全部交给大模型，而是先让确定性分类器给出结论，模型只承担剩余的少数疑难情形。

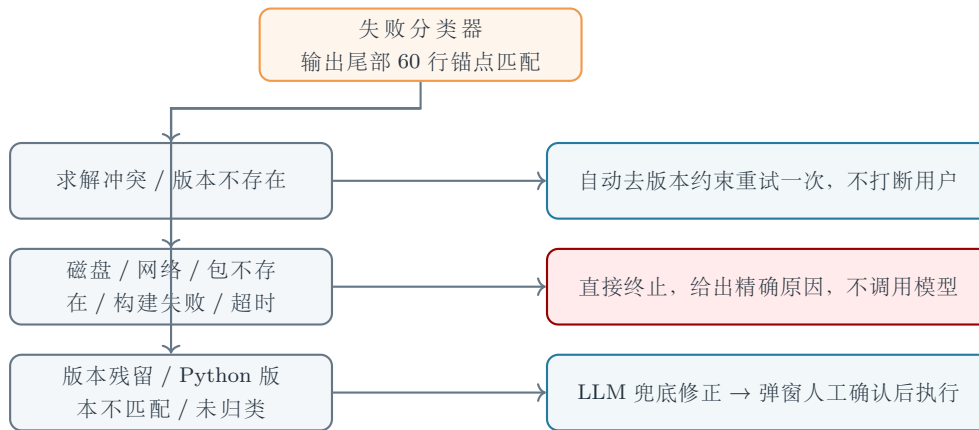
PTY 执行与停滞判定。每条安装命令在 PTY 中执行，与 Web 终端共用同一套机制：conda/pip 自带的下载进度条、颜色和交互提示原样经 SSE 推送到浏览器中的 xterm.js，用户看到的就是真实执行过程，不存在模拟进度。超时判定采用“停滞”语义而非固定总时长——PyTorch 这类包要下载数 GB，慢速网络下健康下载也会持续一小时以上；连续十分钟无任何新输出才判定卡死（进度条刷新即是存活的证据），另设四小时总时长兜底，两者均可通过环境变量调整。超时不进入失败分类器、不喂给模型：模型修不了“时间不够”，直接终止并明示原因，重试会利用 pip/conda 缓存显著加快。

失败分类器。命令失败时，分类器在输出尾部 60 行内做锚点匹配，把失败归为八类：磁盘/权限、网络、包不存在、构建失败、求解冲突、版本不存在、Python 版本不匹配、未归类，并提取缺失包名、冲突上下文、HTTP URL 等结构化信息。以尾部为主要匹配面，是为了避开 pip 构建日志中部的网络重试警告，防止把健康的重试过程误判为网络故障。

分层处置策略。处置按“确定性优先”排序：

1. **确定性自动修复：**求解冲突和版本不存在先自动去掉版本约束重试一次，让 conda/pip 求解器自选兼容组合，全程不打断用户。原失败命令被标记为“已被取代”，最终成败只看重试结果，不因历史失败误报。
2. **直接终止：**磁盘、网络、包不存在、构建失败和超时属于模型修不了的问题（模型无法凭空变出磁盘空间或编译器），系统给出精确原因立即结束，不浪费一轮模型调用，也不让模型猜测不存在的包名。
3. **LLM 兜底修正，人工确认后执行：**其余失败（版本残留、Python 版本不匹配、未归类）交给模型。系统把真实软件源查询结果（可用版本与构建列表）、集群硬件上下文和目标环境 Python 版本一并注入提示词，要求模型只输出修正后的安装命令，版本号必须来自查询结果；服务端再做一道确定性校验——若模型给 py3.10 环境选了 py311 构建，整体拒绝。修正方案以弹窗形式提交用户确认：确认前安装流程挂起等待，终止安装操作也能唤醒该等待，消除“用户在弹窗出现前点了终止”的竞态。

三层策略的入口与边界如下图所示——分类结果决定走哪条路，模型只出现在第三条：



终止与残留防护。无论超时还是用户手动终止，进程都不直接 SIGKILL：先发送 SIGINT/SIGHUP 让 pip 停止写入 site-packages、conda 中止事务并标记未解压包，等待六秒后仍存活才强制终止，降低留下残缺环境的概率。中断的安装支持增量重试：已成功执行的命令可跳过，只重跑失败及未执行部分。

安装后验证与反馈闭环。全部命令结束后，系统用一次 `conda list` 快照逐一核验勾选选项是否真的进入目标环境，而非以命令退出码为准。作业失败日志中被确定性抽取的缺失依赖会写回项目需求记录，参与下一次“检查依赖”分析，形成安装层面的运行反馈闭环。

9.4 共享登录节点上的环境与并发

服务环境、项目计算环境、Conda base 必须隔离；同时初始化同一项目可能破坏前缀。系统为每个项目创建独立前缀，生成自适应 `activate.sh`，用线程锁覆盖 FastAPI 线程池并用 `flock` 覆盖多进程，且对中断半成品进行自愈处理。

9.5 跨生命周期上下文闭环

一个作业可能排队数小时，HTTP 请求早已结束。系统将作业注册为可观察记录，独立追踪终态，再把报告写回原项目/运行目录会话。失败日志中确定性抽取的缺失依赖还会反馈到下一次依赖分析，形成“准备 - 提交 - 运行 - 诊断 - 修正”的闭环。

9.6 浏览器终端的系统编程细节

普通 subprocess 管道不能正确承载交互式 shell、ANSI 控制和窗口大小变化。系统通过 PTY 提供真实终端语义，在 `asyncio` 与阻塞读取线程之间安全转发字节，并处理 SIGWINCH、Origin、并发配额和断连后的进程回收。这一模块涉及浏览器协议、异步 I/O 和 Unix 进程控制的协同。

10 创新点

- 模型与执行权解耦的受控提交。**模型只产出意图和命令正文，后端锁定所有关键调度字段、环境与日志，并在提交后用 TRES/GRES 证据反向核验资源申请是否生效。

2. **面向具体平台的模型校准。**系统提示词内置在该集群实测确认的平台事实（分区名称区分大小写、分区—账户—QoS 对应关系、默认配额、scrontab 禁用）与数据口径纪律（当前队列以 `list_jobs` 实时逐条统计为准，`get_diag` 的累计统计禁止混作当前状态），使模型回答贴合本平台实际而非通用 HPC 印象。
3. **带证据链的知识检索。**平台文档按标题分块并向量化（磁盘缓存、低于相似度阈值丢弃），回答保留来源与相似度；向量服务不可用时降级为关键词检索，构成“证据优先”原则在知识侧的落地。
4. **版本感知的依赖预检与失败分类自愈。**依赖在安装前先与真实软件源逐项比对，针对“把其它集群 module 版本号当 conda 版本”的高频错误按同 major.minor → 同 major → 最新版的三级回退给出建议版本；安装失败经确定性分类器归为八类，常见冲突自动去版本重试，环境类失败直接终止并明示原因，仅少数疑难情形交由模型修正且必须经用户确认；超时判定采用“停滞”语义而非固定总时长。
5. **运行反馈驱动的环境闭环。**作业失败日志中确定性抽取的缺失依赖写回项目需求；多运行目录共享项目依赖而会话隔离，报告自动回流原会话，使环境知识随实际使用逐步补全。

11 部署与运行设计

11.1 运行环境

要求 Python 3.10+、Conda、可访问 Slurm REST API 与 OpenAI 兼容模型服务，并部署在可执行 Slurm CLI 的登录节点。服务端依赖主要包括 FastAPI、Uvicorn、OpenAI SDK、Requests、NumPy、Pydantic、WebSockets、ptyprocess 与 Packaging。前端为单页原生 Web，不需要 Node 构建链。

11.2 启动与访问链路

```
# remote login node
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
export LLM_API_KEY="..."
./start-server.sh

# local ~/.ssh/config
Host 107.ustc.edu.cn
    LocalForward 8080 /home/<user>/slurm-ai-agent/server.sock
```

本地浏览器访问 `http://localhost:8080`，请求经 SSH 隧道转发到远端 Unix Socket。健康检查可通过 `curl --unix-socket server.sock http://localhost/health` 完成。

12 测试与验收建议

测试域	关键用例	通过标准
功能	创建项目、上传、依赖检查/安装、生成正文、提交、监控、查看报告	主流程无断点且状态可恢复
资源校验	无授权账户、QoS 越界、CPU 分区申请 GPU、超时限	提交前明确拒绝并指出字段
路径安全	绝对路径、...、重复运行目录、缺失入口/配置	越界拒绝，常见路径错误可定位到行
模型工具	多工具连续调用、工具异常、无工具纯问答	上下文结构合法，错误不被掩盖
Slurm 兼容	JWT 过期、服务端忽略过滤参数、GPU TRES 延迟可见	单次刷新重试，客户端过滤，核验状态准确
依赖并发	同项目并行初始化/安装、中断半成品	串行化成功且可自愈
终端	Origin 不匹配、超过 4 会话、调整尺寸、异常断连	拒绝策略与进程清理符合预期
降级	Embedding 不可用、LLM 报告失败	关键词结果或原始日志报告仍可用

13 可演进方向

系统仍有四条明确的演进主线。

智能体交互的流式化。当前智能体在一个工具调用循环结束后才整体回显结果，长流程下的等待感明显。后续可借鉴 DeepSeek 等前沿系统在 harness 层的流式设计，把回复文本、工具调用意图与工具执行进度逐步流式回显到前端：模型边生成边呈现，工具执行状态实时可见，既缩短感知延迟，也让“模型正在做什么”对用户全程透明。这一改动仅涉及展示层与事件通道，不改变既有的工具权限与校验边界。

依赖选择的可视化。当前依赖确认是一份平铺的勾选清单，能表达“装不装”，但难以表达“有哪些可选”。预检阶段其实已经拿到了真实软件源的结构化数据（渠道、版本列表、构建变体），后续可把它转化为可视化的选择界面：以包为根节点，向外发散展示不同 channel（conda-forge、bioconda 等）、不同版本与构建（nompi/cuda、py310/py311）的可选分支，让“同名包在不同渠道的差异”“版本与构建的兼容关系”一目了然；交互形式既可以保留命令行式的补全与筛选，也可以做成发散图浏览。由于数据全部来自现有 conda search/pip index 预检结果，该改动只增强呈现与选择方式，不扩大执行边界，最终装什么仍由用户逐项确认。

终端便捷性的持续提升。Web 终端目前提供的是“可用”的登录 shell，后续可向“好用”演进。一是常在与可达性：会话与 PTY 解耦持久化（tmux 式），浏览器关闭或网络中断后重新打开即可恢复现场，长时运行的交互进程不因页面刷新而丢失。二是输入便捷化：借鉴 fish 等现代 shell 的体验，基于命令历史提供灰字建议、参数补全与语法高亮。三是与超算场景特征绑定：补全候选不只来自通用历史，还结合系统已掌握的上下文——Slurm 命令的常用参数、本项目路径与 Conda 环境名、个人历史作业的提交命令与模板变量，让“上次是怎么提交的”一次按键即可找回。这些增强均发生在用户自己的 shell 会话内，不改变终端的权限边界与并发限制。

面向更多学科场景的验证扩展。当前端到端流程已在深度学习训练等场景验证，后续将系统性地跑通更多应用场景：覆盖物理、化学、材料、生物等学科的常用计算软件与公开数据集（如 GROMACS 分子动力学、第一性原理计算、Job Array 批量任务等），为每类场景沉淀模板、依赖清单与验收用例，形成可回归的端到端测试集。这既能暴露各学科软件在依赖求解、许可文件与并行构建上的差异，也为依赖别名库、模板库和报告指标的完善提供真实输入。

在此基础上，其余工程改进包括：自动化单元/集成测试与 OpenAPI 契约测试；为作业监控引入更稳定的任务队列；在文档规模扩大后接入 reranker；建立依赖包名与 import 名映射库；对报告增加资源效率指标；对高风险文件操作增加回收站机制；在不扩大模型权限的前提下，将受控工具扩展到更多平台能力。

14 结论

Slurm AI Agent 的价值不只是为 Slurm 加一个聊天界面，而是围绕 HPC 作业生命周期建立了一套“语义理解可智能、执行边界可确定、运行结果可追溯”的工程体系。通过项目化状态、Function Calling、REST/CLI 分层、受控脚本生成、资源回读核验和反馈式依赖修正，系统在易用性、安全性与真实性之间取得平衡，适合教学、科研训练和批量实验等共享算力场景。